

design and train a convolutional neural network for Image classification for domain of your choice

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
```

```
(X_train, y_train), (X_test, y_test) = keras.datasets.fashion_mnist.load_data()
```

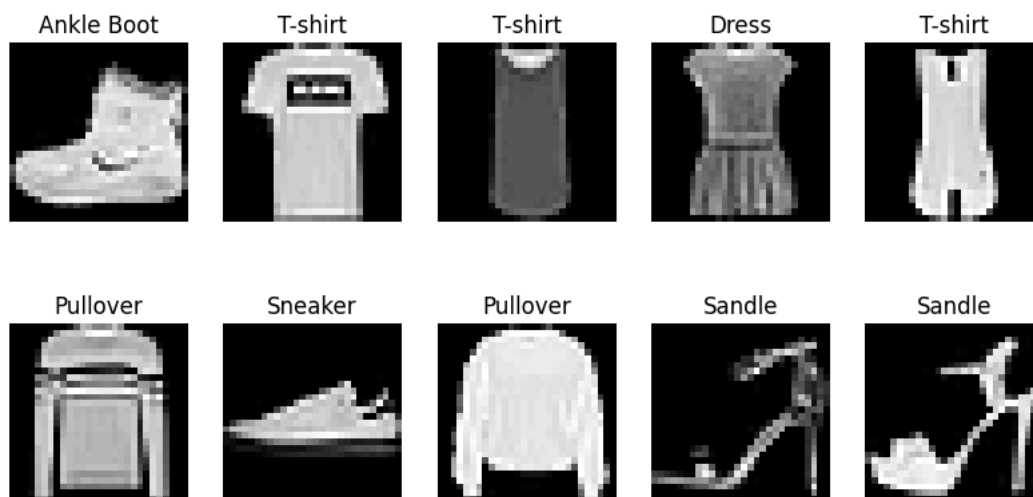
```
X_train, X_test = X_train / 255.0, X_test / 255.0
```

```
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)
```

```
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

```
class_names = ['T-shirt', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandle', 'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']
```

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_train[i].reshape(28, 28), cmap='gray')
    plt.title(class_names[np.argmax(y_train[i])])
    plt.axis('off')
plt.show()
```



```
model = keras.Sequential([
    keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(28, 28, 1)),
    keras.layers.MaxPooling2D(2,2),
    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),
    keras.layers.Flatten(),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(10, activation='softmax')
])
```



```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape` to
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

```
history = model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=5, batch_size=64, verbose=1)
```

```
Epoch 1/5
938/938 — 59s 61ms/step - accuracy: 0.7226 - loss: 0.7713 - val_accuracy: 0.8635 - val_loss: 0.3789
Epoch 2/5
938/938 — 75s 54ms/step - accuracy: 0.8633 - loss: 0.3817 - val_accuracy: 0.8783 - val_loss: 0.3355
Epoch 3/5
938/938 — 88s 61ms/step - accuracy: 0.8812 - loss: 0.3280 - val_accuracy: 0.8860 - val_loss: 0.3092
Epoch 4/5
938/938 — 78s 56ms/step - accuracy: 0.8925 - loss: 0.2927 - val_accuracy: 0.8961 - val_loss: 0.2883
Epoch 5/5
938/938 — 79s 53ms/step - accuracy: 0.9009 - loss: 0.2661 - val_accuracy: 0.8982 - val_loss: 0.2778
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")
model.save('fashion_mnist_cnn.keras')
```

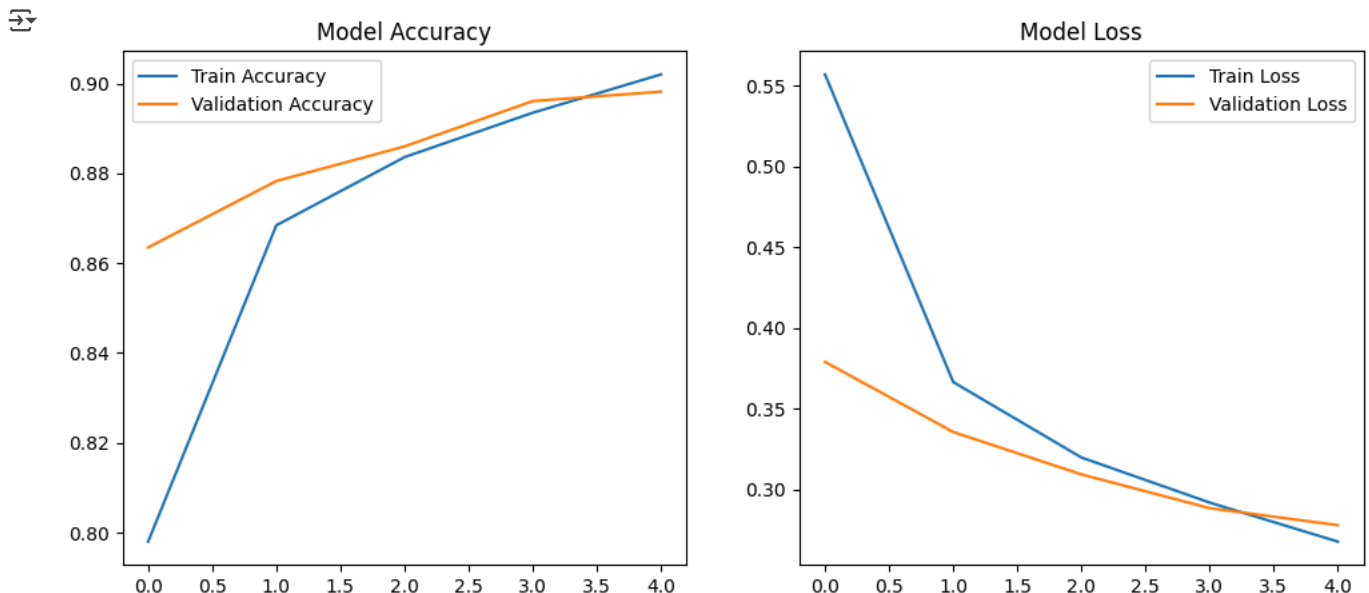
```
313/313 — 4s 11ms/step - accuracy: 0.8958 - loss: 0.2800
Test Accuracy: 0.8982
```

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Model Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Model Loss')

plt.show()
```



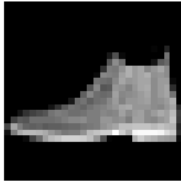
```
predictions = model.predict(X_test)
pred_labels = np.argmax(predictions, axis=1)
```

↗ 313/313 ————— 3s 11ms/step

```
plt.figure(figsize=(10, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_test[i].reshape(28, 28), cmap='gray')
    plt.title(f"Pred: {class_names[pred_labels[i]]}")
    plt.axis('off')
plt.show()
```

↗

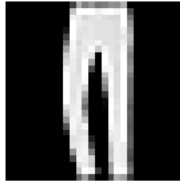
Pred: Ankle Boot



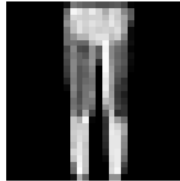
Pred: Pullover



Pred: Trouser



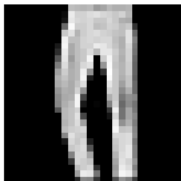
Pred: Trouser



Pred: Shirt



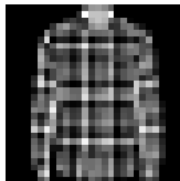
Pred: Trouser



Pred: Coat



Pred: Shirt



Pred: Sandle



Pred: Sneaker

